

YASH YADAV (CU24250185)

Double-click (or enter) to edit

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload an image
uploaded = files.upload()

# Get uploaded filename
filename = list(uploaded.keys())[0]

# Read image in grayscale
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Could not read the image.")
else:
    print("Image loaded successfully!")
    print("Image size:", img.shape)

plt.figure(figsize=(6, 6))
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

 No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving pexels-hakantahmaz-6015200.jpg to pexels-hakantahmaz-6015200.jpg

Image loaded successfully!

Image size: (4000, 6000)

Original Grayscale Image



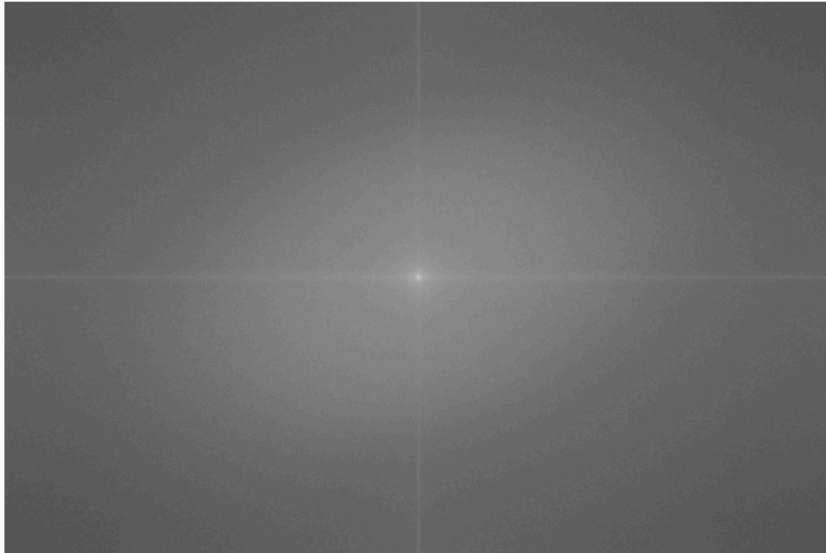
```
# Compute 2D Fourier Transform
dft = np.fft.fft2(img)

# Shift zero frequency component to center
dft_shift = np.fft.fftshift(dft)

# Calculate magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(dft_shift) + 1)

plt.figure(figsize=(8, 6))
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()
```

Magnitude Spectrum



```

rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# Create a mask
mask_low = np.zeros((rows, cols), np.uint8)

# Radius of low-pass filter
radius = 50

# Create circular low-pass mask
Y, X = np.ogrid[:rows, :cols]
distance = np.sqrt((X - ccol)**2 + (Y - crow)**2)

mask_low[distance <= radius] = 1

# Apply filter
filtered_dft_low = dft_shift * mask_low

# Shift back
dft_ishift_low = np.fft.ifftshift(filtered_dft_low)

# Inverse Fourier Transform
img_low = np.fft.ifft2(dft_ishift_low)

# Get magnitude
img_low = np.abs(img_low)

# Normalize for display
img_low = cv2.normalize(img_low, None, 0, 255, cv2.NORM_MINMAX)
img_low = img_low.astype(np.uint8)

plt.figure(figsize=(6, 6))
plt.imshow(img_low, cmap='gray')
plt.title("Low-Pass Filtered Image")
plt.axis("off")
plt.show()

```

Low-Pass Filtered Image



```

# Create high-pass mask
mask_high = np.ones((rows, cols), np.uint8)

# Remove low-frequency center
mask_high[distance <= radius] = 0

# Apply high-pass filter
filtered_dft_high = dft_shift * mask_high

# Shift back
dft_ishift_high = np.fft.ifftshift(filtered_dft_high)

# Inverse Fourier Transform
img_high = np.fft.ifft2(dft_ishift_high)

# Get magnitude
img_high = np.abs(img_high)

# Normalize
img_high = cv2.normalize(img_high, None, 0, 255, cv2.NORM_MINMAX)
img_high = img_high.astype(np.uint8)

plt.figure(figsize=(6, 6))
plt.imshow(img_high, cmap='gray')
plt.title("High-Pass Filtered Image")
plt.axis("off")
plt.show()

```

High-Pass Filtered Image



```

plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(img_low, cmap='gray')
plt.title("Low-Pass Filtered")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(img_high, cmap='gray')
plt.title("High-Pass Filtered")
plt.axis("off")

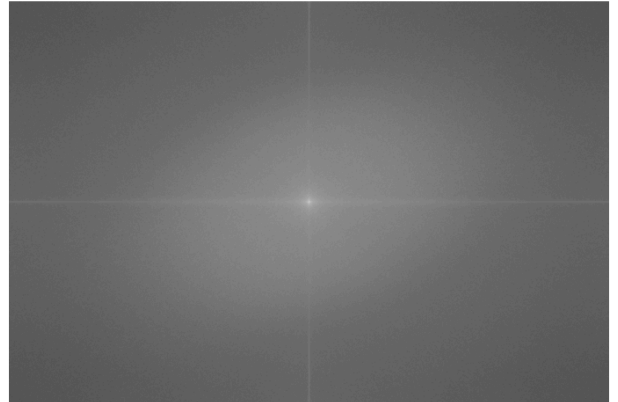
plt.tight_layout()
plt.show()

```

Original Image



Frequency Spectrum



Low-Pass Filtered



High-Pass Filtered

